

SavdoShop

Big Data course - final project report

Student: _____ Group: _____

Public URL: _____

GitHub: _____

Date: 05.11.2026

1. Business idea

Problem. A foreign tourist or a Tashkent local who wants a real souvenir (atlas dress, Margilan ceramics, an Bukhara knife, a chapan) has to walk through several bazaars, prices are not published, and nobody knows whether the item is still in stock. Small masters and shops have no way to show their range online.

Target users. Tourists and city residents aged 20-45 who buy gifts; small shops and masters who sell handicraft. Orders are taken by phone and in Telegram every day in Uzbekistan - an online catalogue with a real stock check removes the "is it still available?" phone call.

How it earns money. Commission 7-10% on every order paid through the site, plus paid "featured product" placement for shops, plus delivery fee. Revenue per order is visible on the dashboard (Q9, /api/stats/sales-by-city).

Competitors. (1) OLX.uz - classifieds, no stock, no delivery, no reviews; (2) Zoodmall / Uzum - general marketplaces where handmade goods are mixed with mass imports and there is no product story. SavdoShop is a vertical marketplace: verified handmade sellers, real stock, photo + review per product.

2. Architecture

The project uses **polyglot persistence**: two databases, each used for what it is good at, with one Express API in front of both.

PostgreSQL 18 - the money

Customers, products, orders, order_items, payments. Strict schema, foreign keys, CHECK constraints, NUMERIC(12,2) for so'm, and a real transaction: an order either fully exists or does not exist at all.

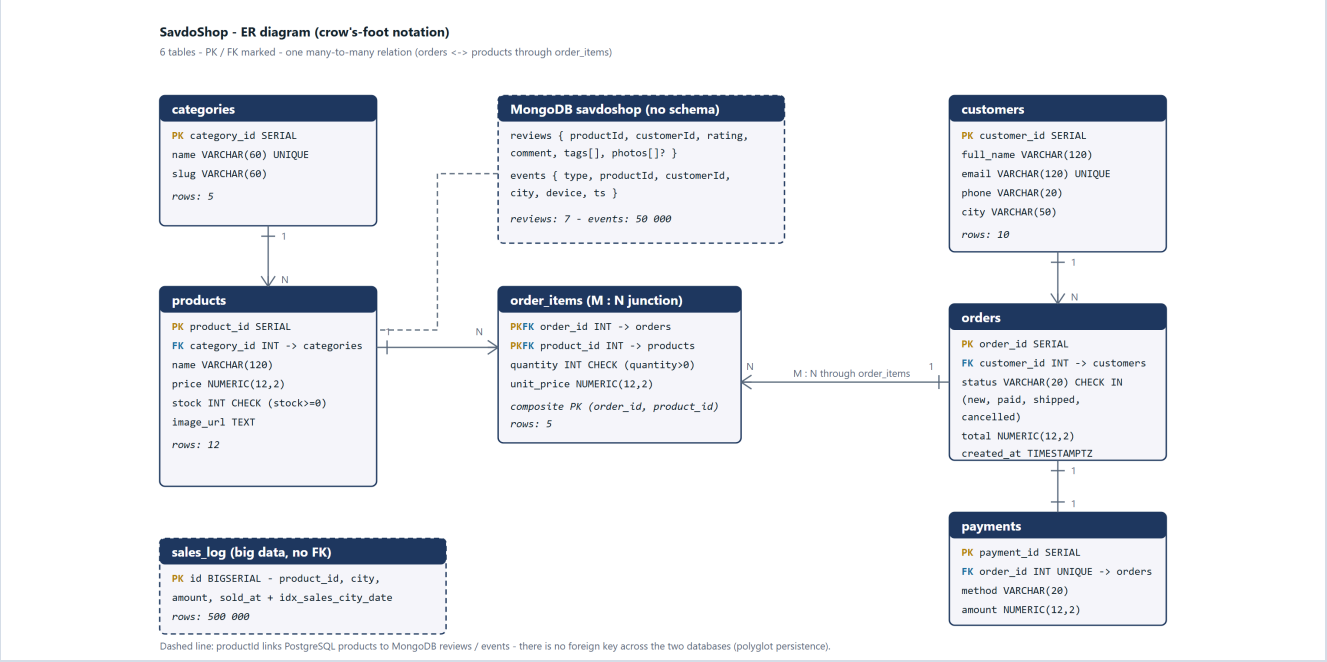
MongoDB 8.0 - the volume

Reviews (some have photos, some do not - no fixed schema needed) and the clickstream events (50 000 documents, growing with every page view). Aggregations feed the dashboard.

Flow: browser (4 HTML pages, app.js) - HTTPS - Express routes - pg.Pool / MongoClient - databases. Deployed as: Render (web service) + Supabase (PostgreSQL) + MongoDB Atlas (M0), optional Cloudflare tunnel for a quick demo link.

Why this split. Money data must never be inconsistent, so it needs ACID and foreign keys. Reviews and events are write-heavy, semi-structured and analysed in batches - a document store fits and does not force a column for an optional "photos" field. One API keeps the front-end simple: it does not know that two databases exist.

3. Database design



6 tables. categories 1:N products, customers 1:N orders, orders 1:N order_items, products 1:N order_items - so orders <-> products is a many-to-many relation resolved by the junction table order_items with a composite primary key. payments is 1:1 with orders (UNIQUE order_id).

Constraints and why

- **NUMERIC(12,2) for money, never FLOAT** - 0.1 + 0.2 must not become 0.30000000000000004 in a price.
- **CHECK (stock >= 0)**, **CHECK (quantity > 0)** - impossible values are refused by the database, not only by the code.
- **CHECK (status IN ('new','paid','shipped','cancelled'))** - a fixed list instead of a lookup table.
- **UNIQUE (email)** on customers, **UNIQUE (order_id)** on payments - one account, one payment per order.
- **ON DELETE CASCADE** on foreign keys - deleting a customer removes their orders and order lines.
- **unit_price is copied into order_items** - prices change (Q3 gives a discount), but an old order must keep the price the customer actually paid. This is the deliberate "denormalisation for history" decision.
- **Indexes on foreign keys**: idx_products_category, idx_orders_customer, idx_orders_created.
- Normalisation: every fact lives in one place - a category name exists only in categories, no comma-separated lists inside a column.

Sample data

02_seed.sql loads 32 realistic rows: 5 categories, 10 customers with Uzbek names and cities, 12 products (atlas dress, chapan, ceramics, knives, power bank) with prices in so'm, 3 orders, 5 order lines, 2 payments. Both scripts run on a fresh database without errors (verified by dropping and recreating the schema).

4. Queries and analytics

20 SQL queries in `03_queries.sql`: 4 CRUD, 5 JOIN (one LEFT JOIN), 4 GROUP BY (one HAVING), 1 view, 2 window functions, 1 sub-query, 1 transaction, 2 free. Every query carries its business question as a comment. 14 MongoDB operations in `mongo_queries.js`: 5 CRUD, 5 aggregations, 2 index/explain, 1 listing. Below are the five most interesting SQL and three Mongo ones; the full output of all 34 is in week1_results.pdf.

Q9 - GROUP BY + HAVING: which cities bring real money?

```
SELECT cu.city, SUM(o.total) AS revenue, COUNT(*) AS orders
FROM orders o
JOIN customers cu ON cu.customer_id = o.customer_id
WHERE o.status <> 'cancelled'
GROUP BY cu.city
HAVING SUM(o.total) > 500000
ORDER BY revenue DESC;
```

city	revenue	orders
Tashkent	3230000.00	2
Samarkand	515000.00	1

Q7 - LEFT JOIN + IS NULL: which customers have never ordered?

```
SELECT cu.full_name, cu.city
FROM customers cu
LEFT JOIN orders o ON o.customer_id = cu.customer_id
WHERE o.order_id IS NULL;
```

full_name	city
Sardor Qodirov	Samarkand
Malika Yusupova	Namangan
Timur Saidov	Tashkent
Otabek Ergashev	Andijan

Q12 - window function: price ranking inside each category

```
SELECT category, name, price,
       RANK() OVER (PARTITION BY category ORDER BY price DESC) AS price_rank
FROM v_product_catalog;
```

category	name	price	price_rank
Beauty	Rose face cream	120000.00	1
Beauty	Argan hair oil	95000.00	2
Clothing	Winter chapan	950000.00	1
Clothing	Atlas silk dress	650000.00	2
Clothing	Cotton shirt	180000.00	3

Q17 - window function: how a customer's spending grows (running total)

```
SELECT o.customer_id, cu.full_name, o.order_id, o.created_at, o.total,
       SUM(o.total) OVER (PARTITION BY o.customer_id ORDER BY o.created_at) AS running_total
FROM orders o
JOIN customers cu ON cu.customer_id = o.customer_id
ORDER BY o.customer_id, o.created_at;
```

customer_id	full_name	order_id	created_at	total	running_total
1	Aziza Karimova	1	2026-11-04T22:02:56.730Z	830000.00	830000.00
2	Bekzod Tursunov	2	2026-11-04T22:02:56.730Z	515000.00	515000.00
3	Dilnoza Rahimova	4	2026-11-04T22:02:57.204Z	378000.00	378000.00
4	Jasur Aliyev	3	2026-11-04T22:02:56.730Z	2400000.00	2400000.00

Q13 - transaction: place an order and reduce stock atomically

```
BEGIN;
   INSERT INTO orders (customer_id) VALUES (3) RETURNING order_id;
   INSERT INTO order_items (order_id, product_id, quantity, unit_price)
   SELECT currval('orders_order_id_seq'), product_id, 1, price
   FROM products WHERE product_id = 4;
   UPDATE products SET stock = stock - 1 WHERE product_id = 4;
   UPDATE orders SET total = (SELECT SUM(quantity * unit_price) FROM order_items
                              WHERE order_id = currval('orders_order_id_seq'))
   WHERE order_id = currval('orders_order_id_seq');
COMMIT; -- or ROLLBACK; to cancel everything
```

Four statements, one commit. If any step fails, ROLLBACK leaves no trace: no half-written order, no lost stock. The API route `POST /api/orders` does exactly this with one pooled client and `finally { client.release() }`.

M6 - average rating per product (MongoDB)

```
db.reviews.aggregate([
  { $group: { _id: "$productId", avgRating: { $avg: "$rating" }, reviews: { $sum: 1 } } },
  { $sort: { avgRating: -1 } }
])
```

```
[
  {
    _id: 10,
    avgRating: 5,
    reviews: 1
  },
  {
    _id: 1,
    avgRating: 4.5,
    reviews: 2
  },
  {
    _id: 4,
    avgRating: 4,
    reviews: 2
  },
  {
    _id: 7,
    avgRating: 4,
    reviews: 1
  },
  {
    _id: 11,
    avgRating: 2,
    reviews: 1
  }
]
```

M8 - top 5 most viewed products in the last 30 days

```
[
  {
    _id: 12,
    views: 838
  },
```

```
  {
    _id: 11,
    views: 828
  },
  {
    _id: 8,
    views: 821
  },
  {
    _id: 5,
    views: 816
  },
  {
    _id: 1,
    views: 810
  }
]
```

M11 - most common review tags ($unwind turns an array into rows)

```
[
  {
    _id: 'quality',
    count: 3
  },
  {
    _id: 'size',
    count: 2
  },
  {
    _id: 'delivery',
    count: 1
  },
  {
    _id: 'comfort',
    count: 1
  },
  {
    _id: 'gift',
    count: 1
  },
  {
    _id: 'battery',
    count: 1
  }
]
```

5. Big data experiment (500 000 rows)

`04_big_data.sql` creates **sales_log** and fills it in one statement with `generate_series(1, 500000)`: a random product, a random city from an array, a random amount and a random date inside the last year. Insert took **34.1 s**, then `ANALYZE` refreshed the planner statistics.

The query

```
EXPLAIN ANALYZE
SELECT COUNT(*), SUM(amount) FROM sales_log
WHERE city = 'Khiva' AND sold_at > now() - interval '30 days';
```

Without an index

1110 ms

Seq Scan: PostgreSQL read the whole table and threw away 164,970 rows.

With idx_sales_city_date (city, sold_at)

82 ms

Bitmap Index Scan + Bitmap Heap Scan: it jumped straight to the matching city and date range. About **13.5x faster** on this PC.

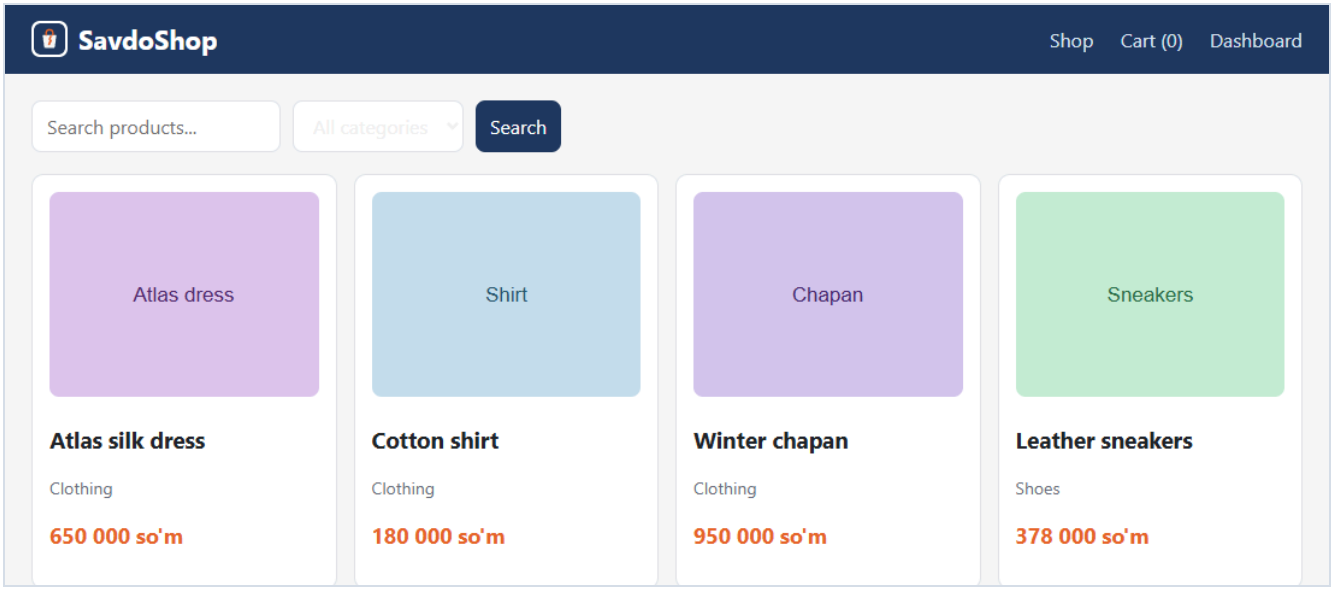
Variant	Scan node	Planning	Execution	Rows read
no index	Seq Scan	3.084 ms	1110.1 ms	all 500 000
composite index	Bitmap Index Scan	13.353 ms	82.4 ms	only the matching ones

Explanation. The composite index is ordered by city first and then by date, so a filter on both columns becomes a range lookup instead of a full read. Indexes are not free: they take disk space and slow down INSERTs, because every new row must also be written into the index - so we index only columns used in WHERE / JOIN. The same experiment in MongoDB (M12 vs M13) on 50 000 events: before the compound index `{city:1, type:1}` the query returned **887** documents but examined **7144** in 47 ms; after the index it examined **887** - exactly what it returned - in 30 ms.

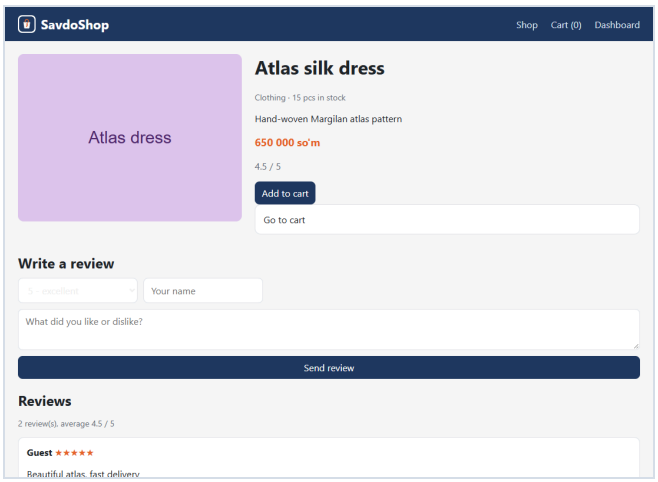
Time series and pre-computed summary

Monthly revenue over the whole year (Step 4, 1096 ms) and a **materialized view** mv_city_daily (city, day, sales, revenue): a normal view would re-scan 500 000 rows for every dashboard request, a materialized view stores about 2 900 summary rows and is refreshed with `REFRESH MATERIALIZED VIEW` when new data arrives. Step 6 (top 3 products in every city with `ROW_NUMBER() OVER (PARTITION BY city ...)`) runs in 837 ms over the same 500 000 rows.

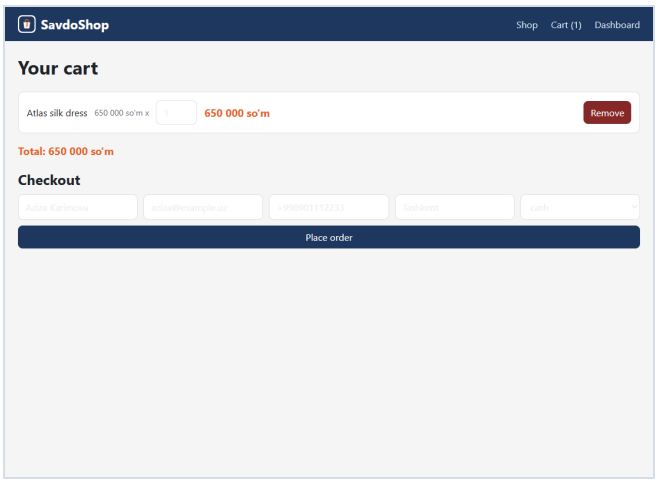
6. Website and API



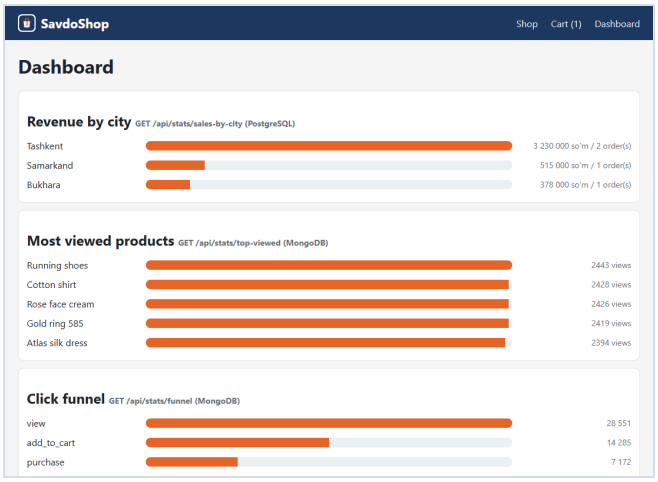
Home page: 12 products from GET /api/products, search and category filter.



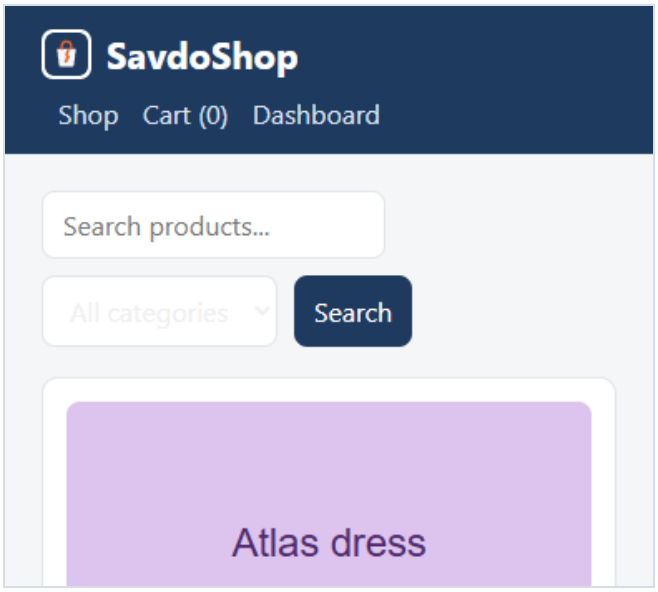
Product page: price and stock from PostgreSQL, reviews from MongoDB.



Cart from localStorage before checkout.



Dashboard: revenue by city (SQL), top viewed and funnel (Mongo), connection health.



The same page at 390 px: one card per row (CSS grid auto-fill minmax(240px, 1fr)).

API routes

Method	URL	Purpose	Database
GET	/api/health	checks both connections, returns server time and review count	both
GET	/api/categories	category list for the filter	PostgreSQL
GET	/api/products?q=&category=&page=	search, filter, 12 per page	PostgreSQL
GET	/api/products/:id	product + its reviews; also writes a "view" event	both
POST	/api/products/:id/reviews	save a review (validates rating 1-5, length)	MongoDB
POST	/api/orders	transaction: stock check, order, order_items, payment, events	both
GET	/api/customers/:id/orders	order history with json_agg of the lines	PostgreSQL
GET	/api/stats/sales-by-city	revenue per city	PostgreSQL
GET	/api/stats/top-viewed	top 5 viewed products, names joined from SQL	both
GET	/api/stats/funnel	view / add_to_cart / purchase counts	MongoDB

The transaction in the API

`POST /api/orders` takes one client from the pool, runs BEGIN, checks stock with `UPDATE products SET stock = stock - $q WHERE product_id = $id AND stock >= $q RETURNING stock`. If a line returns no row, the item is out of stock: the code throws, the whole transaction is rolled back and the API answers **409 Conflict** with "Not enough stock for product N". Two customers cannot buy the same last item: the second UPDATE simply matches zero rows. On success the order, the order_items lines, the payment and the MongoDB purchase events are committed together, and `finally { client.release() }` always returns the connection to the pool.

7. Deployment and testing

Hosting choices

- **Render** free web service - runs `npm start`, Root Directory = `server`; sleeps after 15 idle minutes, so the first visit takes about a minute.
- **Supabase** free PostgreSQL - the Session pooler string is used (not "Direct connection") and `PGSSL=true`.
- **MongoDB Atlas** M0 - IP access 0.0.0.0/0, the password is URL-encoded because it contains special characters.
- **Cloudflare tunnel** (`cloudflared tunnel --url http://localhost:3000`) - only for a live demo, never as permanent hosting.
- Secrets live only in environment variables (locally `server/.env`, on Render the Environment tab). `.env` is in `.gitignore`.

Test plan (run on the live site in a real browser, clicks automated with Playwright)

#	Test case	Expected	Actual result	Pass
T1	Home page loads	Product list visible	12 cards rendered, 0 console errors	**yes**
T2	Search "silk"	Only matching items	1 card (Atlas silk dress)	**yes**
T3	Filter category = Shoes	Only that category	3 cards	**yes**
T4	Detail page	Data from PostgreSQL + reviews from MongoDB	"Atlas silk dress", 650 000 so'm, 3 reviews	**yes**
T5	Create (review + order)	Success message, new rows	review saved, order #7 created, cart cleared	**yes**
T6	Business rule: stock	Friendly error, nothing saved	409 "Not enough stock for product 10", stock unchanged, no order row	**yes**
T7	Validation	Error message, no crash	empty/short fields rejected by the server with 400	**yes**
T8	Security - XSS	Posted as text, not markup	hi shown literally (esc() before innerHTML)	**yes**
T9	Dashboard	Numbers from SQL and Mongo aggregations	10 bars, funnel view/add_to_cart/purchase, health line	**yes**
T10	Console	No red errors	none after the favicon fix	**yes**

Security and quality checklist

- No passwords in the code or on GitHub - the repository was searched for "password"; only `.env.example` with placeholders is committed.
- Every SQL query uses $1, $2 parameters - no string concatenation, so SQL injection is impossible.
- User text is escaped with esc() before innerHTML; the API also trims and limits the comment length.
- Server validates input: required fields, rating between 1 and 5, quantity > 0, product must exist.
- Money is NUMERIC, never FLOAT. Transactions use one client and always release() it.
- README.md explains how to run the project locally and what each route does.

Problems solved during the project

- **Lost PostgreSQL superuser password** after the winget install (it never asks): reset through a temporary `trust` line in `pg_hba.conf`, `ALTER ROLE postgres PASSWORD ...`, then restored scram-sha-256. The password exists only in `server/.env`.
- **Cyrillic password + Git Bash**: libpq reads PGPASSWORD in the ANSI codepage, so psql reported "password authentication failed" although the password was right. Verified through the Node driver instead; SQL files are UTF-8 without BOM with `PGCLIENTENCODING=UTF8`.
- **Prices arriving as text** ("650000.00") - NUMERIC is returned as a string by pg, so the front-end uses Number(p.price) before formatting.
- **Empty pages** - the boot block in app.js now shows the real error text instead of failing silently.

8. Startup roadmap

Next features

- Accounts and login (bcrypt password hashes), an admin page for sellers to add and edit products, order status tracking for the customer.
- Payment integration (Payme / Click) writing into the existing payments table, and Telegram notifications for new orders - the shop already receives orders by phone today.
- Photos per product (Object Storage), a seller rating on the product page (already collected in MongoDB reviews), and search with synonyms in Uzbek and Russian.

How to scale

- PostgreSQL: read replicas for the catalogue, partition sales_log by month, and keep only the last 12 months hot.
- MongoDB: the events collection is already append-only and shardable by customerId; move old events to cold storage.
- Add Redis for the dashboard results and a queue (Bull) so a purchase writes events asynchronously instead of inside the transaction.
- CDN for images; move the API to two services (shop + analytics) when the dashboard starts to slow the checkout.

What we would analyse with 1 million users

- Funnel by city and device (already the M7 shape at 50 000 events) to find where orders are lost - view to cart to purchase.
- Time-to-first-purchase and repeat rate per acquisition channel; the weekday revenue table (Q18) shows which days need promotion.
- Review text mining: tags that predict a return ("size", "battery") - the product page can then show the real size chart.
- Price elasticity: for each product, revenue by price band, to set the discount that maximises margin instead of volume.